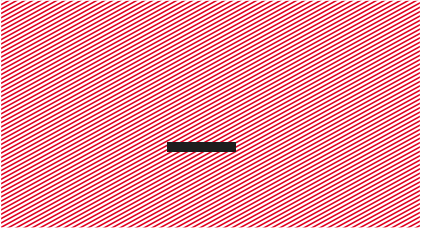


— IoT Devices

Les 2 – Basis IO met HAL-bibliotheek





Waarom HAL-bibliotheken?

Waarom HAL?

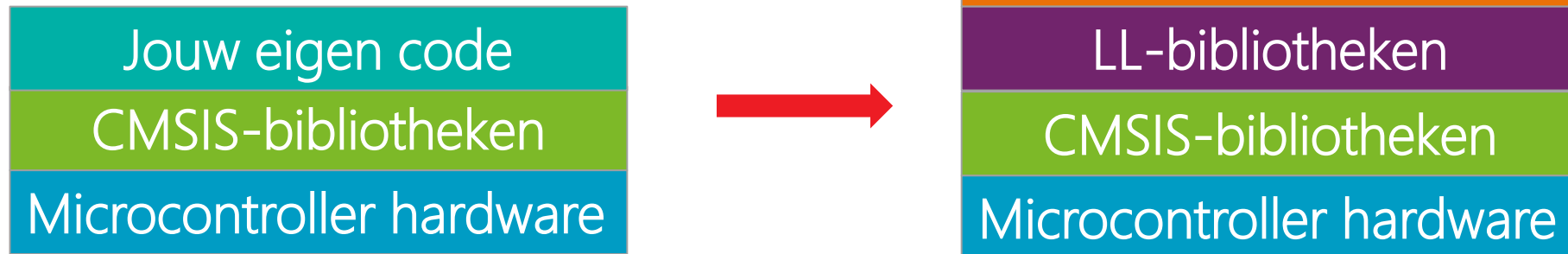
- In het verleden werkten we met CMSIS-code. Die manier van werken is **heel goed**, omdat er een standaard manier van werken gehanteerd wordt: Cortex Microcontroller Software Interface Standard.
- Je kan er heel efficiënt tot **op bit-niveau in de registers** zaken gaan aanpassen.

```
// Clock voor GPIOA inschakelen.  
RCC->AHB2ENR = RCC->AHB2ENR | RCC_AHB2ENR_GPIOAEN;
```

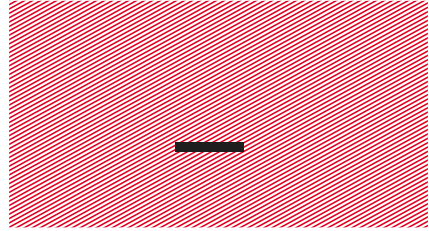
- **Echter**, veranderen van type microcontroller is niet zo eenvoudig...

Waarom HAL?

- Indien we **vlotter van microcontroller willen veranderen**, kan een Hardware Abstraction Layer (HAL) een voordeel bieden.
- De HAL-bibliotheek is een softwarelaag (C-taal) die **bovenop** de CMSIS-bibliotheken ligt, waardoor je onafhankelijk(er) wordt van het type microcontroller...



- **Mogelijks gevaar:** het maakt code soms wel minder efficiënt...



HAL-info?



- Waar kan ik **info** vinden over de HAL-bibliotheek?
 - Er is een goed uitgewerkte PDF met daarin de essentie...
 - https://www.st.com/resource/en/user_manual/um1884-description-of-stm32l4l4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

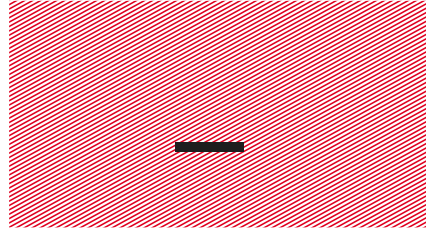


UM1884

User manual

Description of STM32L4/L4+ HAL and low-layer drivers

- Gebruik een goede PDF-lezer om te navigeren via de bladwijzers. Per 'peripheral' is er een hoofdstuk.



HAL demo:
één knop – één LED



HAL demo: één knop – één LED

- Start van het sjabloon '**Nucleo-L476RG – Template**' om één drukknop in te lezen en daarmee één LED aan te sturen...
- Open het **STM32CubeMX-bestand** en blader door de belangrijkste instellingen (System Core en Clock Configuration).
- Open het Keil **µVision** project en stel de firmware op... Maak twee versies:
 - met gebruik van de gekende functies '*SW1Active()*'.
 - met gebruik van de HAL-functies '*HAL_GPIO_ReadPin()*'.



HAL demo: één knop – één LED

```
111 // // Versie 1: met gebruik van de gekende functies (die de HAL-functies 'wrappen').
112 // if(SW1Active())
113 //     ByteToLeds(0x01);
114 // else
115 //     ByteToLeds(0x00);
116
117 // Versie 2: met direct gebruik van de HAL-functies.
118 if(HAL_GPIO_ReadPin(SW1_GPIO_Port, SW1_Pin) == GPIO_PIN_RESET)
119     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
120 else
121     HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
122
123 // OPM: de tweede versie manipuleert enkel één LED, terwijl de eerste versie alle LED's instelt...
124 //     Versie één kan echter ook op die manier uitgewerkt worden indien gewenst.
```

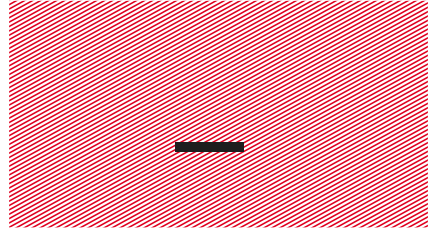
Als je dus in STM32CubeMX de LED's en knoppen verplaatst/aanpast, kan je vrij snel je project klaar maken voor andere hardware (of microcontroller)... Zoiets heet men soms '**migreren**'.

HAL demo: één knop – één LED

Navigeer doorheen het Keil μ Vision project en zoek uit wat er allemaal gebeurt tijdens het opstarten...

Neem daarvoor je tijd! In principe moet ieder lijn code duidelijk zijn.

```
97  /* Initialize all configured peripherals */
98  MX_GPIO_Init();
99  MX_ADC1_Init();
100 MX_USART2_UART_Init();
101 /* USER CODE BEGIN 2 */
102 // Laten weten dat we opgestart zijn, via de USART2 (USB).
103 strcpy(text, "Reboot\r\n");
104 printf("%s", text);
105 /* USER CODE END 2 */
```

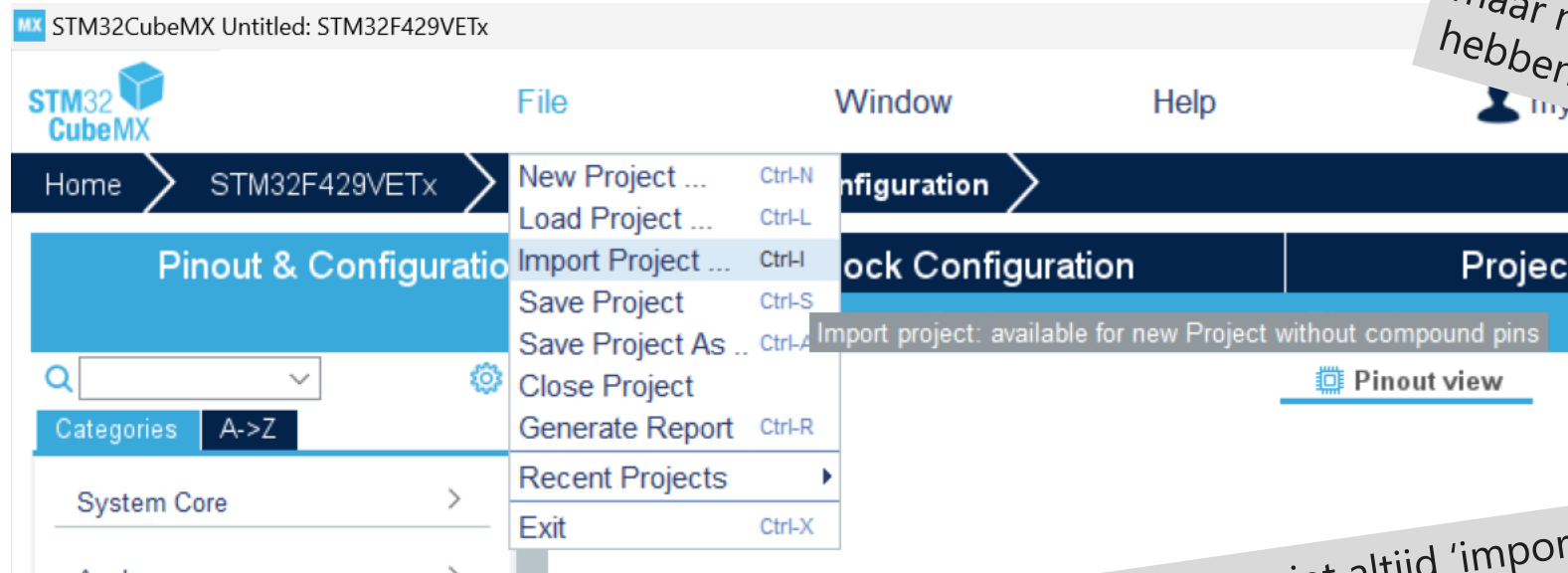


Migreren



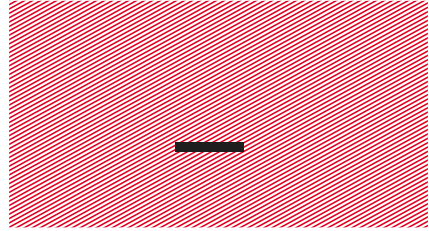
Migreren

- Wil je effectief van de ene naar de andere microcontroller migreren, dan kan je dat met behulp van STM32CubeMX grotendeels automatiseren.
- Gebruik daarvoor de 'import project' optie...



Dit is voorlopig niet onze eerste zorg. We leren nu werken met de Nucleo waar we nog maar net voor gekozen hebben...

Let op: je kan niet altijd 'importeren'.



HAL demo: AD-converter en UART



- Start van het sjabloon '**Nucleo-L476RG – Template**' om de AD-waarde van de on board potentiometer in te lezen en de waarde op de LED's te zetten (via niveau aanduiding).
- **Focus** hierbij op de instellingen van de AD-converter in STM32CubeMX...

HAL demo: AD-converter en UART2

The screenshot displays the STM32CubeMX software interface for a Nucleo-L476RG board. The 'Pinout & Configuration' tab is active, showing the 'ADC1 Mode and Configuration' settings. A red arrow points to the 'ADC1' option in the left-hand 'Categories' list. The 'ADC1 Mode and Configuration' panel shows the 'Mode' section with 'IN5' selected as 'IN5 Single-ended'. The 'Configuration' section shows 'NVIC Settings', 'DMA Settings', 'GPIO Settings', 'Parameter Settings', and 'User Constants' all checked. The 'Search Signals' section shows a search for 'ADC1_IN5' resulting in a table entry for 'PA0'.

Pin Na...	Signal on ...	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	Fast M
PA0	ADC1_IN5	n/a	Analog mo...	No pull-up ...	n/a	n/a

The 'Pinout view' on the right shows the physical pinout of the STM32L476RG LQFP64 package. A red arrow points to the 'POT' pin, which is connected to 'PA0'. Other pins shown include 'LED1', 'SW4', 'PC0', 'PC1', 'PC2', 'PC3', 'VSSA...', 'VDDA...', 'PA1', 'TX2', 'PA2', 'PA3', 'VSS', 'VDD', 'PA4', 'PA5', 'PA6', 'PA7', 'PC4', and 'PC5'.

HAL demo: AD-converter en UART2

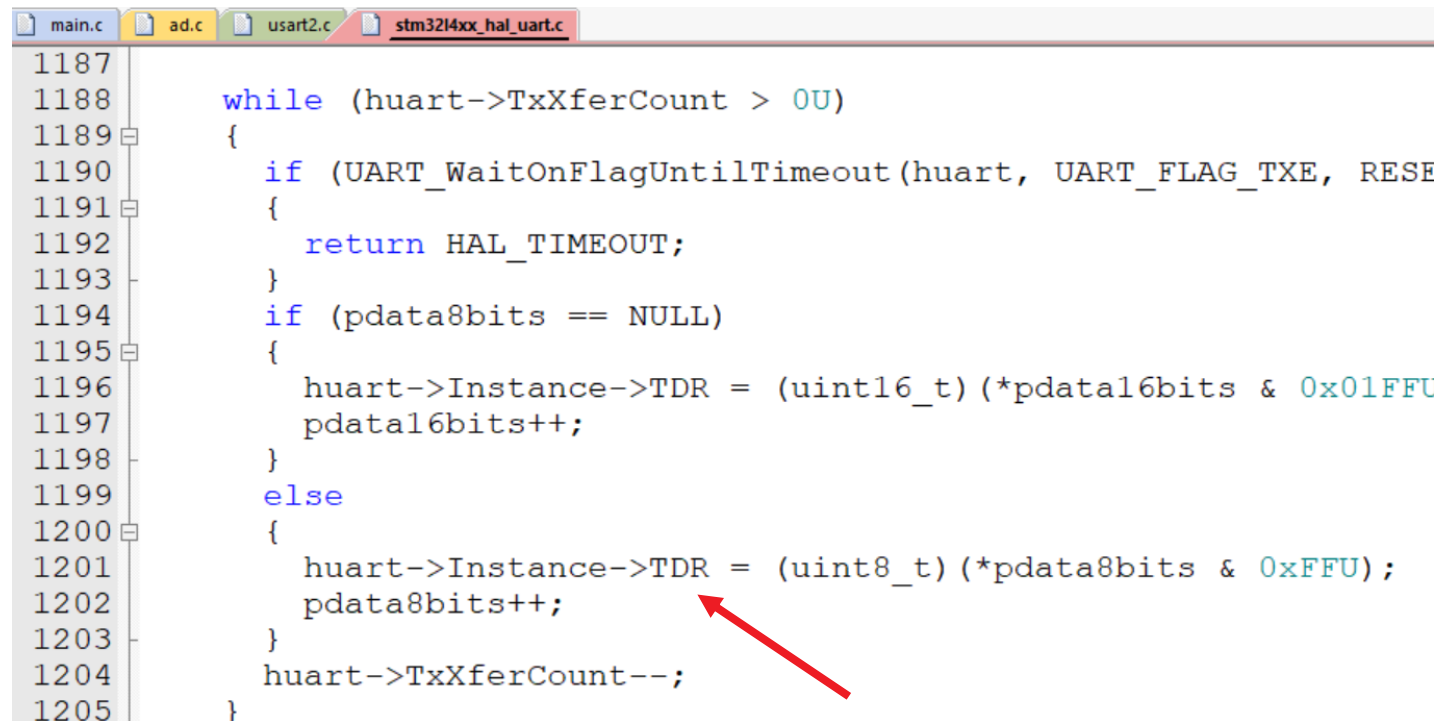
- Pas de firmware aan zoals hieronder. **Zoek uit hoe** alles werkt!

```
106 while (1)
107 {
108     // AD-conversie starten en in een blokkerende functie op het
109     // resultaat wachten.
110     adValue = GetAdValue(&hadcl);
111     adValue = adValue >> 4;    // 12-bit waarde die right aligned was, naar
112                               // 8-bit converteren.
113
114     // Waarde display'n
115     ByteToLevel((uint8_t)adValue);
116
117     // Waarde naar USART2 sturen (USB COM-poort).
118     printf("AD-value: %d.\r\n", adValue);
119
120     // Even wachten
121     HAL_Delay(25);
```

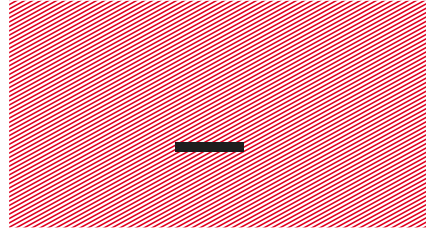
Merk op dat de printf ook hier omgeleid werd (redirect) naar USART2. Dat kan je zien in 'usart2.c'.

HAL demo: AD-converter en UART2

- `printf()` wordt omgeleid naar `HAL_UART_Transmit()`, waarin het **TDR-register opgevuld wordt** met data. Daardoor wordt die byte verzonden over USART2. Zoek onderstaande code op...



```
1187
1188     while (huart->TxXferCount > 0U)
1189     {
1190         if (UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TXE, RESE
1191         {
1192             return HAL_TIMEOUT;
1193         }
1194         if (pdata8bits == NULL)
1195         {
1196             huart->Instance->TDR = (uint16_t)(*pdata16bits & 0x01FFU
1197             pdata16bits++;
1198         }
1199         else
1200         {
1201             huart->Instance->TDR = (uint8_t)(*pdata8bits & 0xFFU);
1202             pdata8bits++;
1203         }
1204         huart->TxXferCount--;
1205     }
```

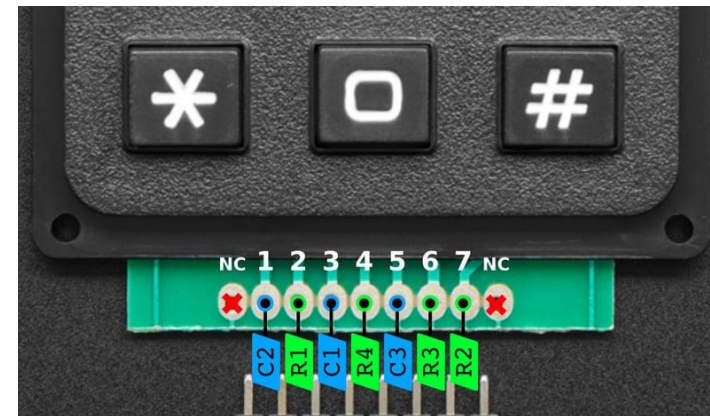


HAL demo: keypad inlezen



HAL demo: keypad inlezen

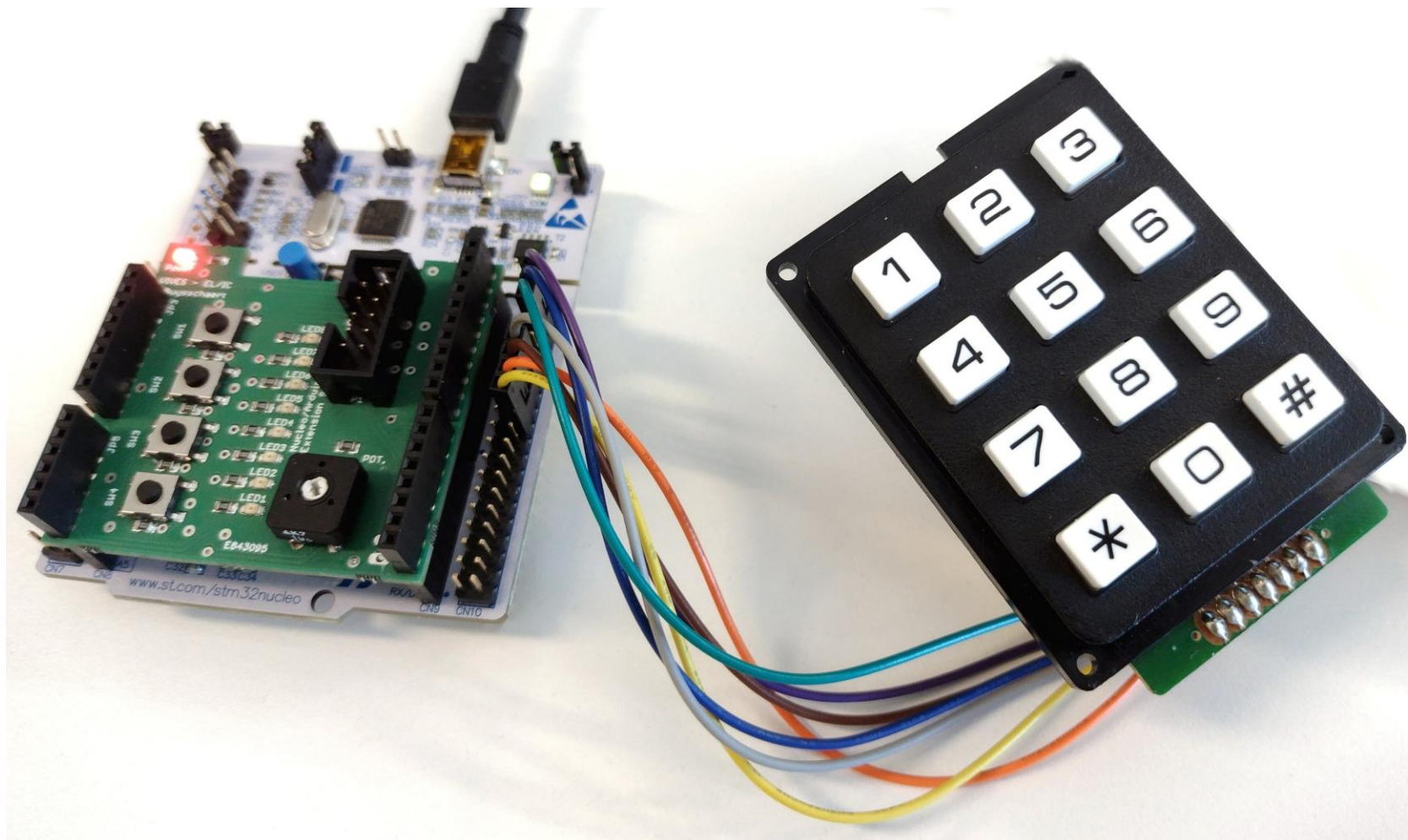
- Lees een **keypad** in van 3 x 4 knoppen.



Afbeeldingen: <https://learn.adafruit.com/matrix-keypad/pinouts>

—

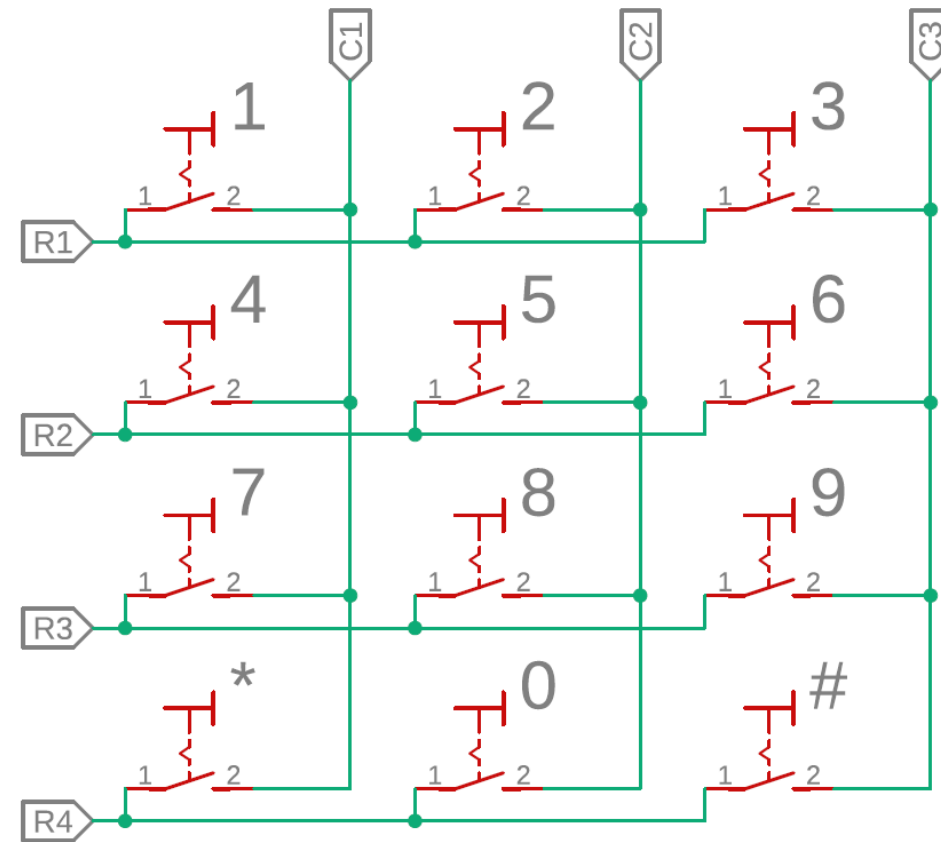
HAL demo: keypad inlezen



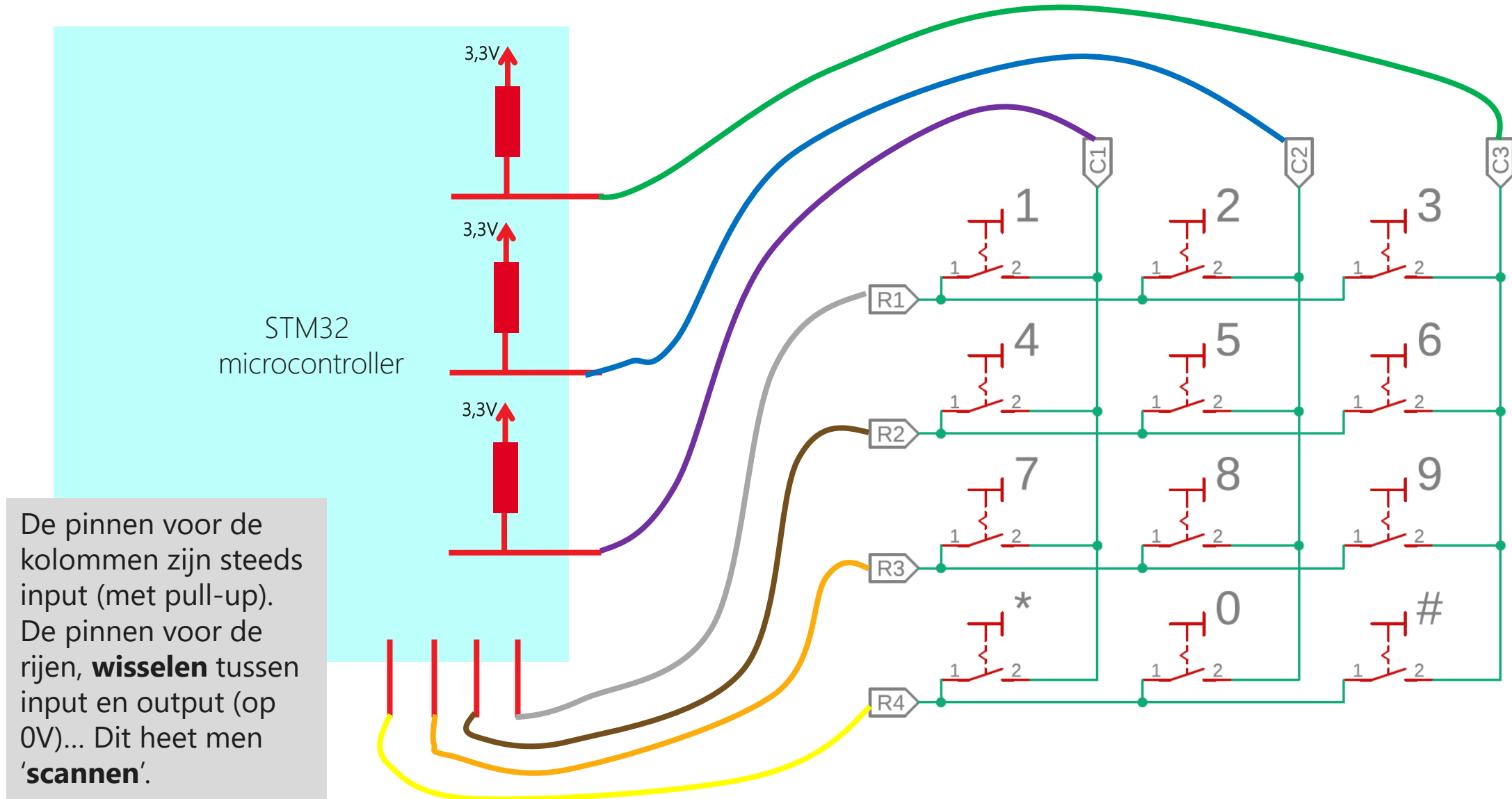


HAL demo: keypad inlezen

- Het interne **schema** ziet er zo uit...
- **Waarom** werkt men zo?
- **Hoe** zou je dat efficiënt kunnen gebruiken?
=> 'scanning'



HAL demo: keypad inlezen



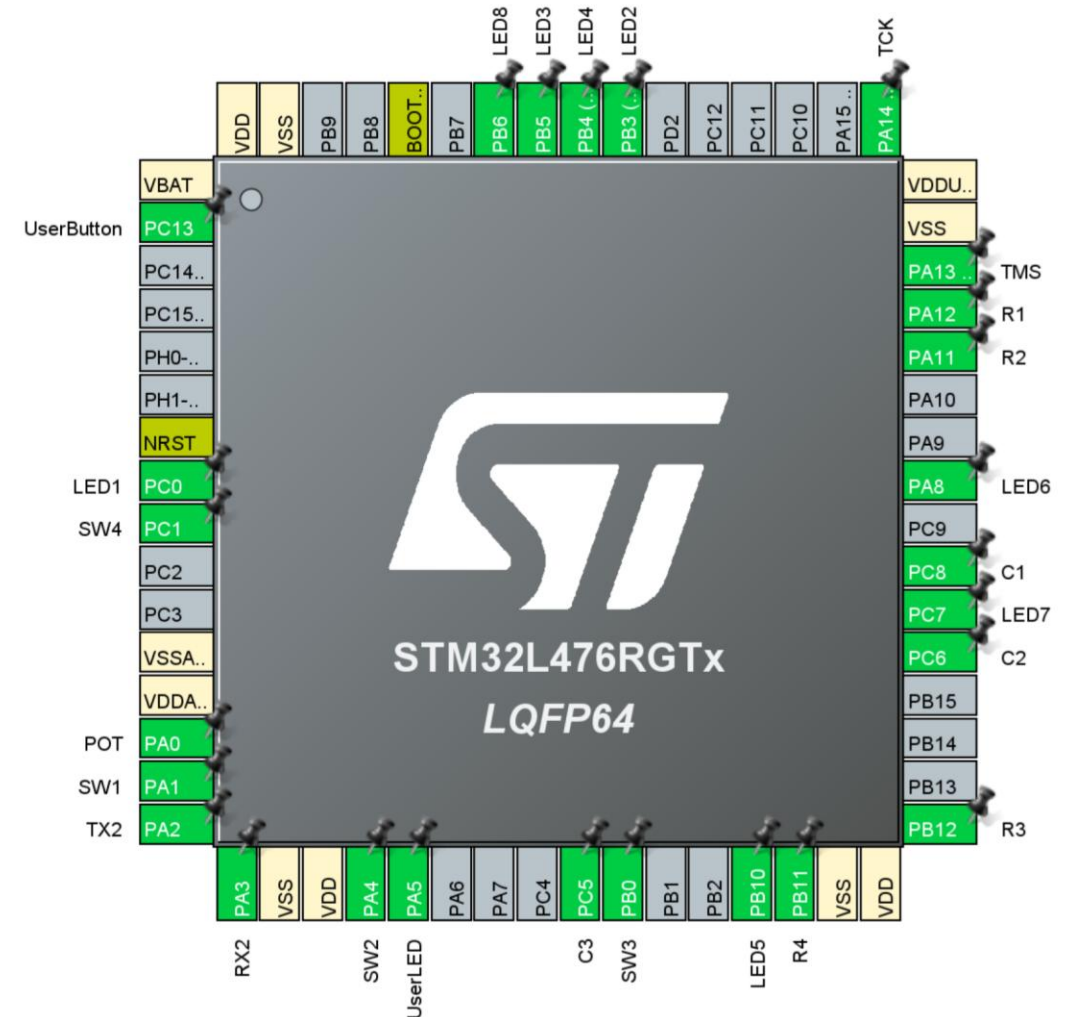
HAL demo: keypad inlezen

Categories
A-Z
System Core
DMA
GPIO
IWDG
NVIC
RCC
SYS
TSC
WWDG
Analog
Timers
Connectivity
Multimedia
Security
Computing
Middleware and Software Packs

Configuration
Group By Peripherals
GPIO
ADC
SYS
USART
Search Signals
Search (Ctrl+F)
Show only Modified Pins

Pin Name	Signal on Pin	GPIO output	GPIO mode	GPIO Pull-up	Maximum ou	Fast Mode	User Label	Modified
PA1	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	SW1	✓
PA4	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	SW2	✓
PA5	n/a	Low	Output Push...	No pull-up an...	Low	n/a	UserLED	✓
PA8	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED6	✓
PA11	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	R2	✓
PA12	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	R1	✓
PB0	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	SW3	✓
PB3 (JTDO-T...	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED2	✓
PB4 (NTRST)	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED4	✓
PB5	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED3	✓
PB6	n/a	Low	Output Push...	No pull-up an...	Low	Disable	LED8	✓
PB10	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED5	✓
PB11	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	R4	✓
PB12	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	R3	✓
PC0	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED1	✓
PC1	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	SW4	✓
PC5	n/a	n/a	Input mode	Pull-up	n/a	n/a	C3	✓
PC6	n/a	n/a	Input mode	Pull-up	n/a	n/a	C2	✓
PC7	n/a	Low	Output Push...	No pull-up an...	Low	n/a	LED7	✓
PC8	n/a	n/a	Input mode	Pull-up	n/a	n/a	C1	✓
PC13	n/a	n/a	Input mode	No pull-up an...	n/a	n/a	UserButton	✓

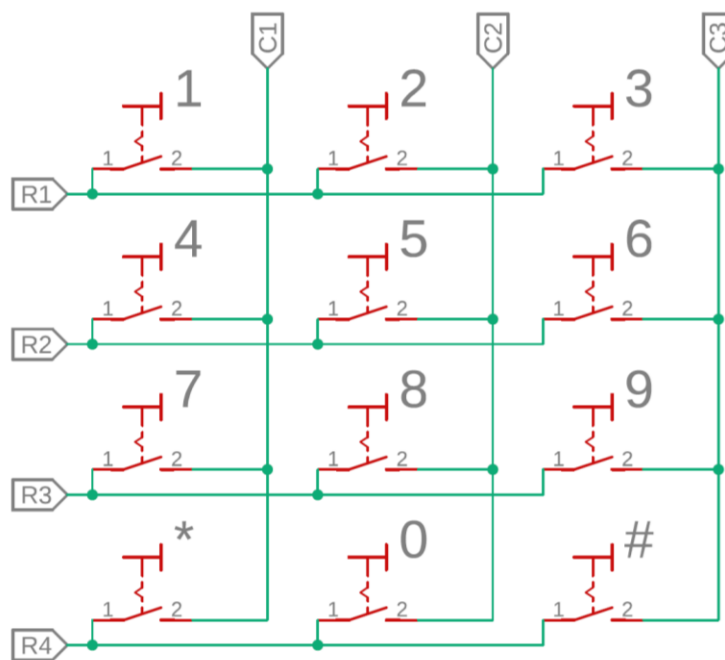
Let op de verwijzing naar de rijen en kolommen van het keypad: R1, R2, ..., C1, ...





HAL demo: keypad inlezen

Indien je wil werken met het codevoorbeeld uit de les, maak dan onderstaande **verbindingen**.



Connection suggestion to Nucleo board:

C1 => PC8
C2 => PC6
C3 => PC5
R1 => PA12
R2 => PA11
R3 => PB12
R4 => PB11



HAL demo: keypad inlezen

- Bekijk de 'scan'-werking.
- Van welk **type** is GPIO_ReInit?

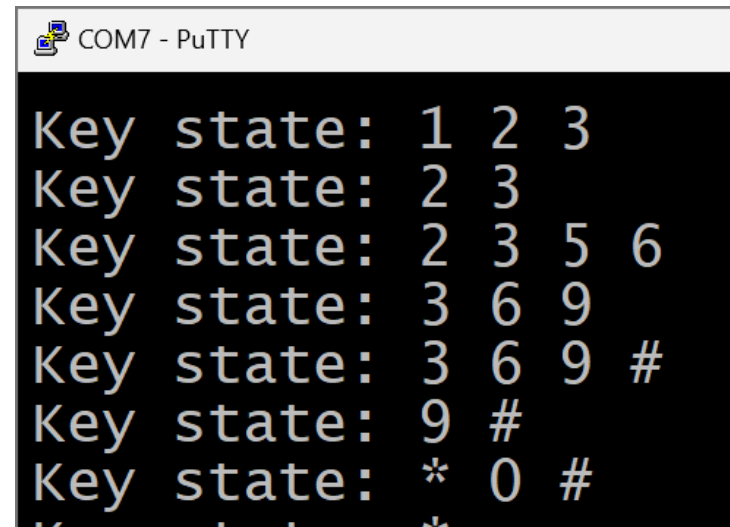
```
26 // Rij 1 als output zetten en laag maken.
27 GPIO_ReInit.Pin = R1_Pin;
28 GPIO_ReInit.Mode = GPIO_MODE_OUTPUT_PP;
29 GPIO_ReInit.Pull = GPIO_NOPULL;
30 GPIO_ReInit.Speed = GPIO_SPEED_FREQ_LOW;
31 HAL_GPIO_Init(R1_GPIO_Port, &GPIO_ReInit);
32 HAL_GPIO_WritePin(R1_GPIO_Port, R1_Pin, GPIO_PIN_RESET);
33
34 // Knop 1 inlezen.
35 if(HAL_GPIO_ReadPin(C1_GPIO_Port, C1_Pin) == GPIO_PIN_RESET)
36     keys |= 0x0001;
37
38 // Knop 2 inlezen.
39 if(HAL_GPIO_ReadPin(C2_GPIO_Port, C2_Pin) == GPIO_PIN_RESET)
40     keys |= 0x0002;
41
42 // Knop 3 inlezen.
43 if(HAL_GPIO_ReadPin(C3_GPIO_Port, C3_Pin) == GPIO_PIN_RESET)
44     keys |= 0x0004;
```



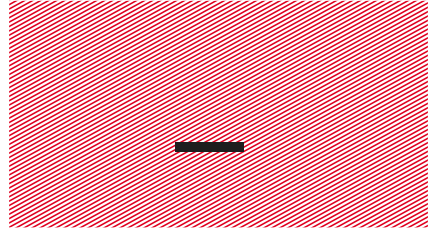
HAL demo: keypad inlezen

Op Toledo is een sjabloon beschikbaar met code om het keypad in te lezen. Gebruik dat **sjabloon** om er later enkele opdrachten mee te maken... Zie: "Nucleo-L476RG - Polling - Keypad – Sjabloon".

```
// Toestand van de knoppen opvragen. Eén bit per knop. Dus 12 bit nodig...  
keys = GetKeys();  
  
// Vul hier jouw eigen code verder aan...
```



```
COM7 - PuTTY  
Key state: 1 2 3  
Key state: 2 3  
Key state: 2 3 5 6  
Key state: 3 6 9  
Key state: 3 6 9 #  
Key state: 9 #  
Key state: * 0 #  
Key state: *
```



HAL demo: 7-segment display aansturen



HAL demo: 7-segment display aansturen

- Stuur twee 7-segment displays aan.

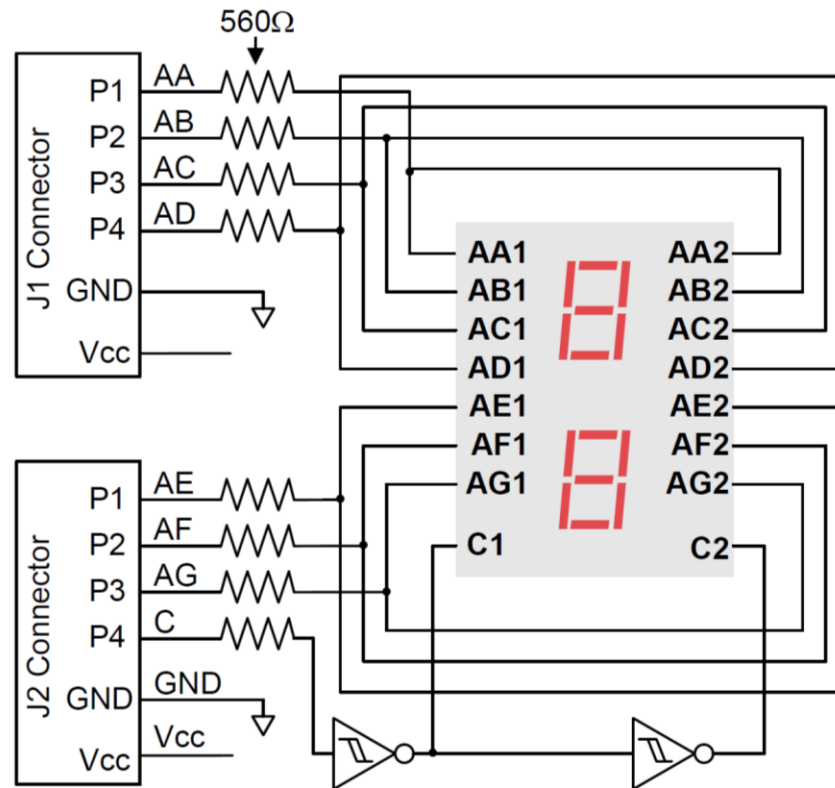


Features include:

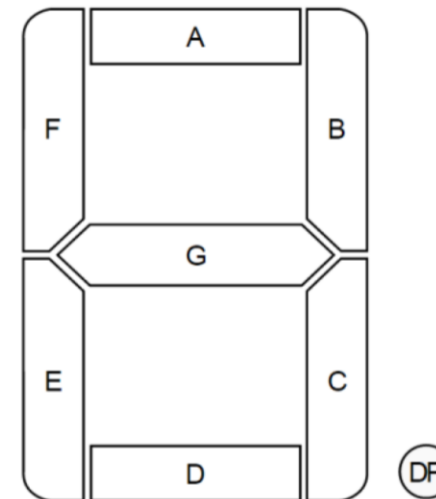
- Two-digit high brightness seven-segment display
- Easily view a counter or timer
- Common Cathode configuration
- Small PCB size for flexible designs 1.0" x 1.7" (2.5 cm x 4.3 cm)
- Two 6-pin Pmod connectors with GPIO interfaces
- Follows [Digilent Pmod Interface Specification](#) Type 1

Afbeelding: <https://digilent.com/reference/pmod/pmodssd/reference-manual>

HAL demo: 7-segment display aansturen



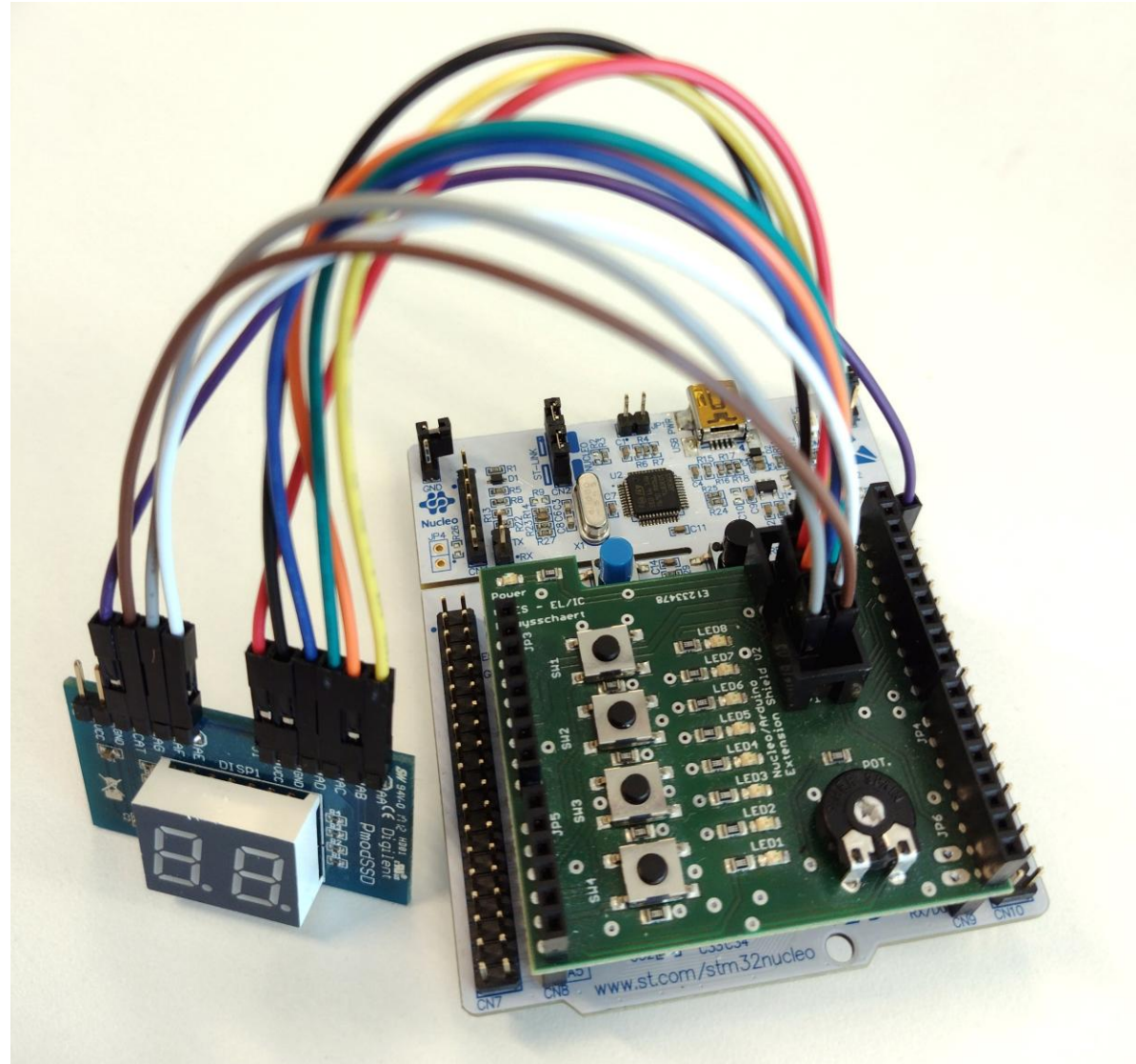
Hoe kan je hier verschillende getallen zetten op de twee displays, als er een koppeling is van de verschillende anodes?
=> multiplexing....



Afbeelding: <https://digilent.com/reference/pmod/pmodssd/reference-manual>

—

HAL demo: 7-segment display aansturen





HAL demo: 7-segment display aansturen

- Bekijk hoe de 'multiplexing' tot stand komt in de code.

```
109 while (1)
110 {
111     // Aansturen van het linker display.
112     SetSegments((count >> 4), left);
113     HAL_Delay(10);
114
115     // Aansturen van het rechter display.
116     SetSegments((count & 0x0F), right);
117     HAL_Delay(10);
```


HAL demo: 7-segment display aansturen

- Bekijk hoe de **tekens** gevormd worden in de code.

```
9 // Segmenten aansturen.
10 switch(data)
11 {
12     case 0:
13         HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, GPIO_PIN_SET);
14         HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, GPIO_PIN_SET);
15         HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, GPIO_PIN_SET);
16         HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, GPIO_PIN_SET);
17         HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, GPIO_PIN_SET);
18         HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, GPIO_PIN_SET);
19         HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, GPIO_PIN_RESET);
20         break;
21
22     case 1:
23         HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, GPIO_PIN_RESET);
24         HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, GPIO_PIN_SET);
25         HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, GPIO_PIN_SET);
26         HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, GPIO_PIN_RESET);
27         HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, GPIO_PIN_RESET);
28         HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, GPIO_PIN_RESET);
29         HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, GPIO_PIN_RESET);
30         break;
```

HAL demo: 7-segment display aansturen

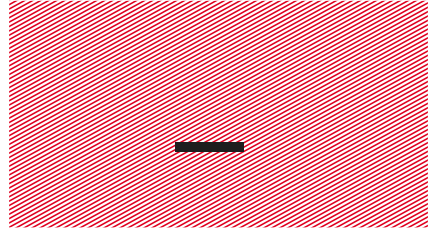
- Bekijk hoe de **tekens** gevormd worden in de code.

```
173 // Linker of rechter display aansturen. Of alles uitdoven.
174 if(display == left)
175     HAL_GPIO_WritePin(CAT_GPIO_Port, CAT_Pin, GPIO_PIN_SET);
176 else
177     if(display == right)
178         HAL_GPIO_WritePin(CAT_GPIO_Port, CAT_Pin, GPIO_PIN_RESET);
179     else
180     {
181         HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, GPIO_PIN_RESET);
182         HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, GPIO_PIN_RESET);
183         HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, GPIO_PIN_RESET);
184         HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, GPIO_PIN_RESET);
185         HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, GPIO_PIN_RESET);
186         HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, GPIO_PIN_RESET);
187         HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, GPIO_PIN_RESET);
188     }
```



HAL demo: keypad inlezen

Op Toledo is een **sjabloon** beschikbaar met code om de 7-segment displays aan te sturen. Gebruik dat sjabloon om er later enkele opdrachten mee te maken... Zie: "Nucleo-L476RG - Pmod SSD (7-segment) – Sjabloon".



Theorieopdracht



Theorieopdracht

Maak gebruik van de demo code voor het **keypad** en de 7-segment **displays** en **combineer** beide items.

Toon welke toets je indrukte op het 7-segment display en verstuur de data ook in **JSON** naar een **WPF-applicatie**.

Zie: **"Theorieopdracht 2 – Keypad, Pmod SSD en JSON"**.